Cinco,Daniela Faith L.
BSIT-3B

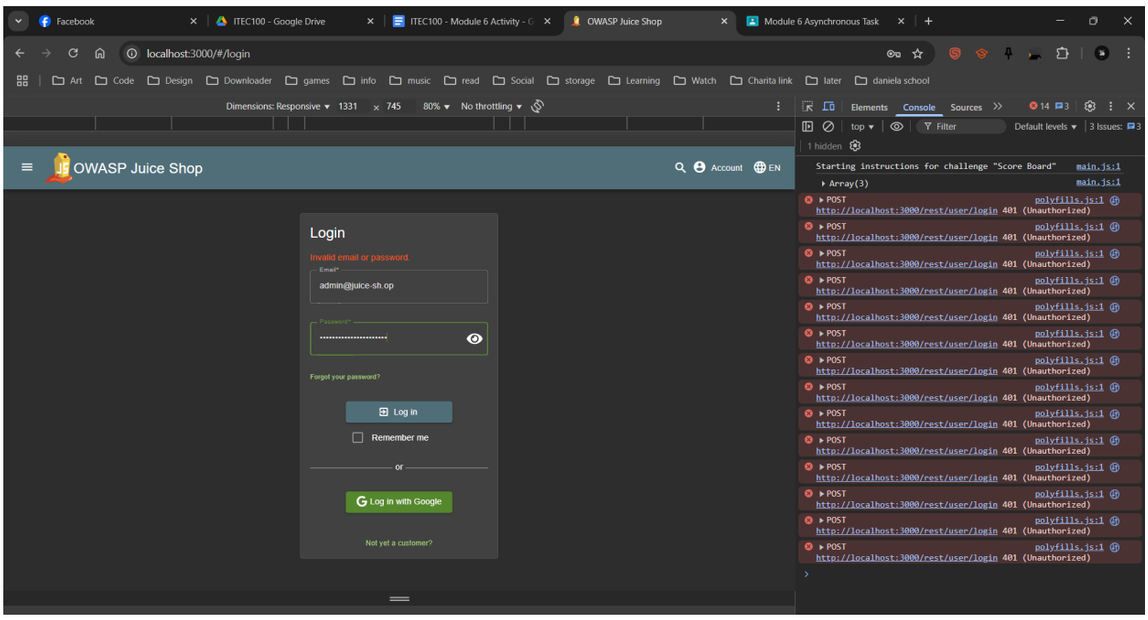
June 1, 2025
Sir. Sarmiento

ITEC100 - Lesson 6 Asynchronous Task

Part 1: Vulnerability Exploration

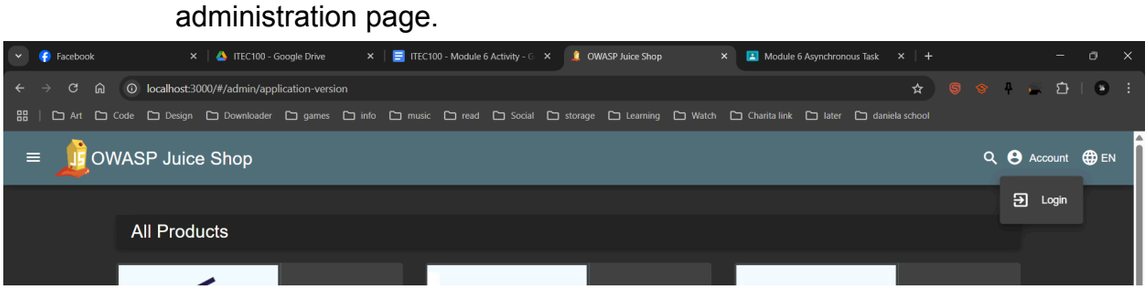
a. Poorly Designed Authentication Mechanism

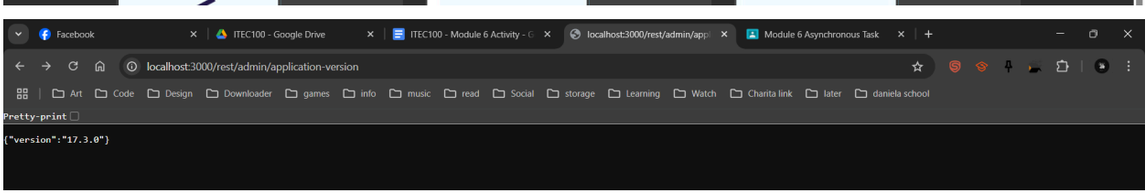
- Login lets users attempt multiple tries of password, allowing brute forcing of password.



b. Inadequate Access Control

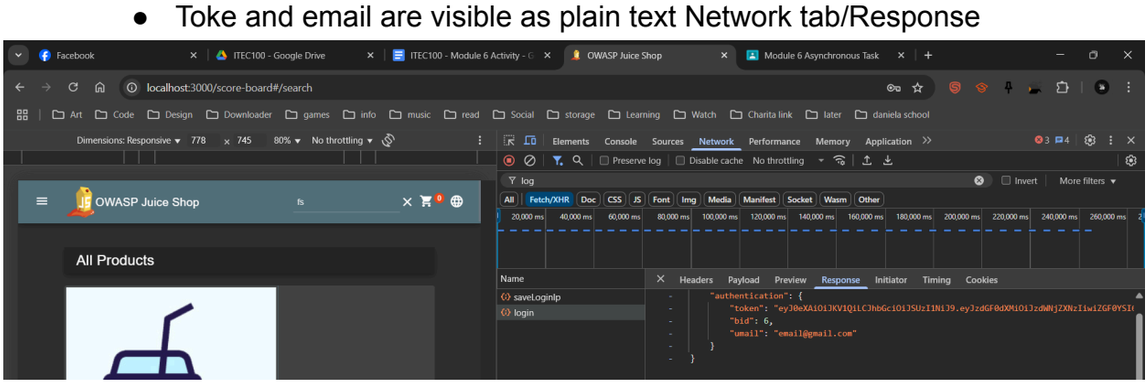
- Admins' features are exposed to regular users when accessing the administration page.





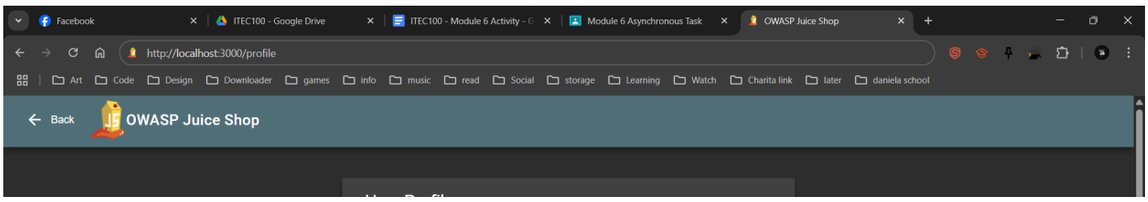
c. Unencrypted Sensitive Data

- Toke and email are visible as plain text Network tab/Response



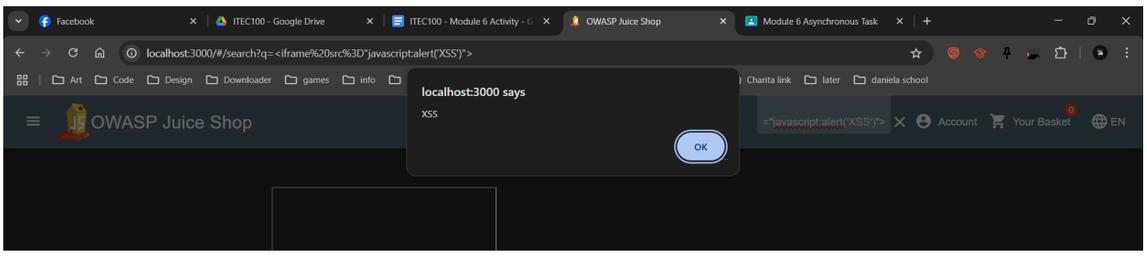
d. Network-related vulnerabilities

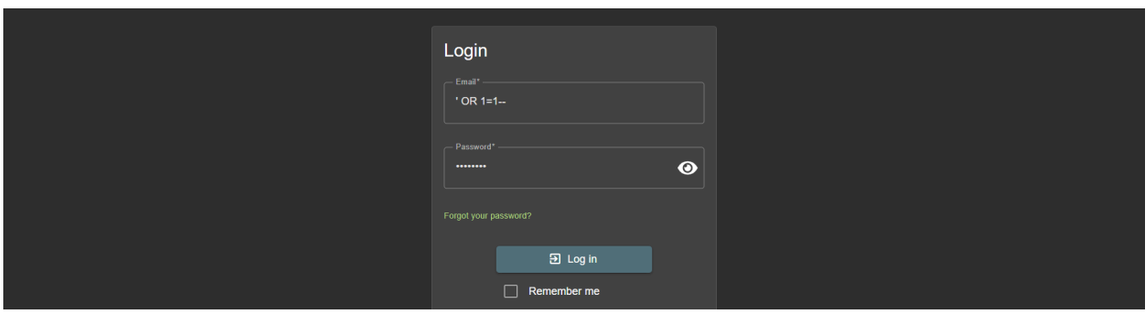
- HTTP is used when sending tokens instead of HTTPS.

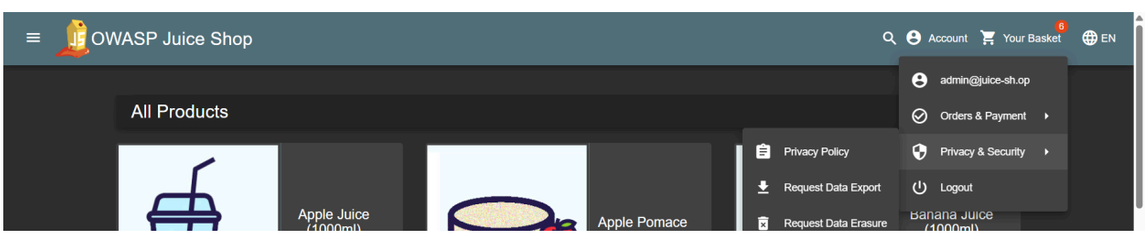


e. Inadequate Input Validation

- User's inputs are not sanitized and directly added to SQL query instead of using prepared statements allowing SQL injection.



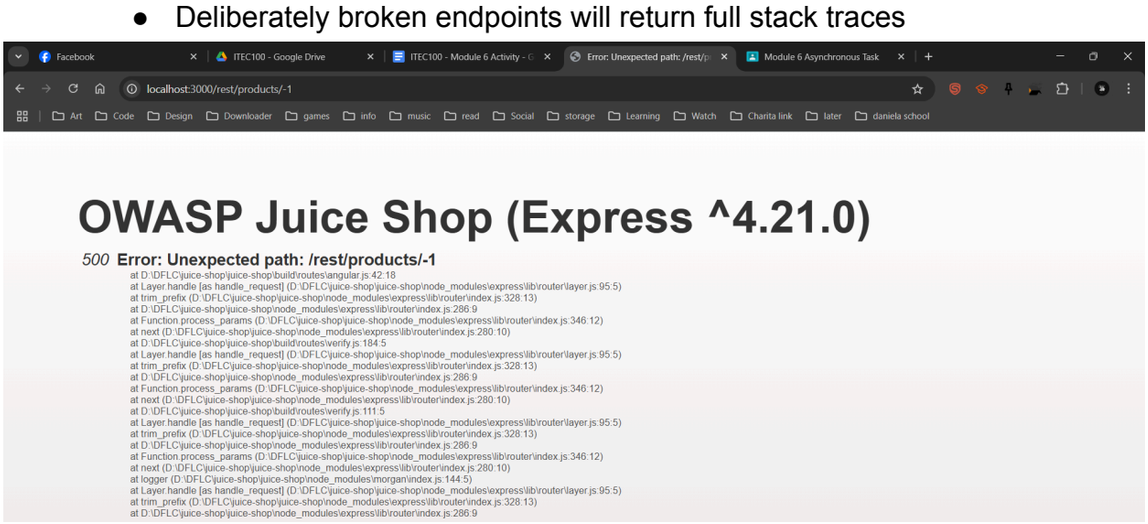




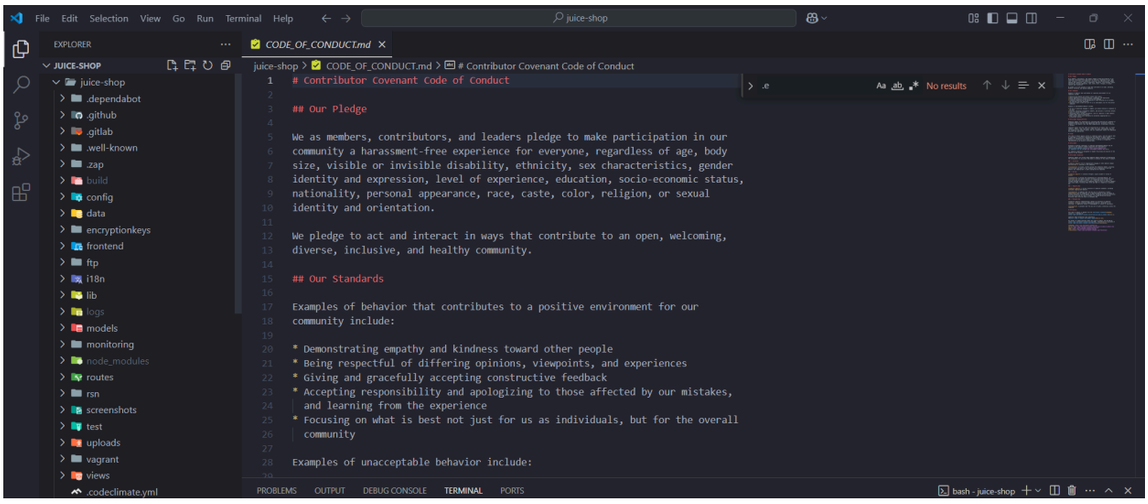

```
return (req: Request, res: Response, next: NextFunction) => {
  if (req.body.email.match(/.*['-;].*/) || req.body.password.match(/.*['-;].*/)) {
    res.status(451).send(res.__('SQL Injection detected.'))
  }
  models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req.body.password || ''
    .then((authenticatedUser) => {
      const user = utils.queryResultToJson(authenticatedUser)
      if (user.data?.id && user.data.totpSecret !== '') {
        res.status(401).json({
          status: 'totp_token_required',
```

f. Error messages leaking technical details

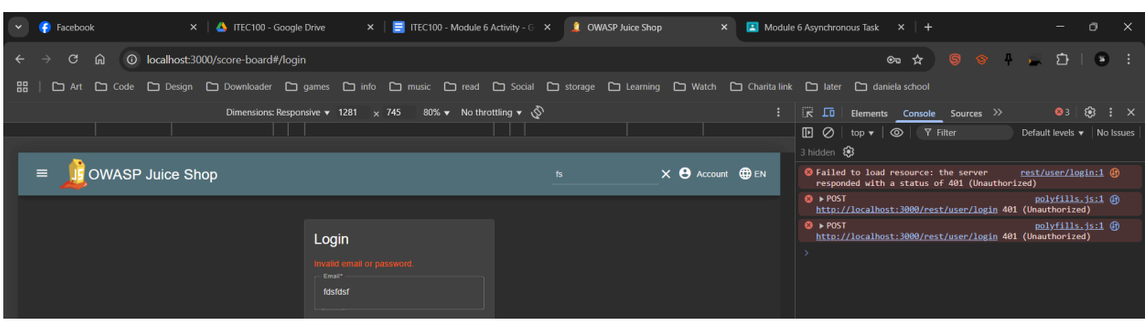
- Deliberately broken endpoints will return full stack traces

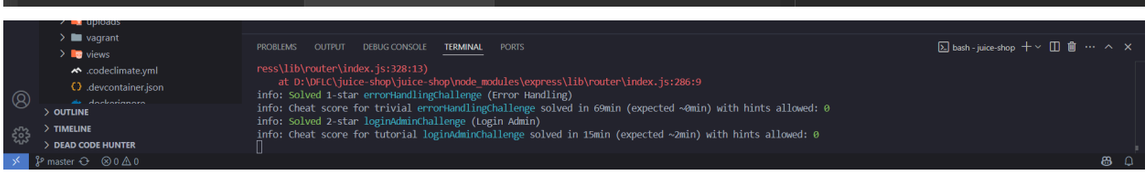


g. **Weak architectural decisions (no API gateway or microservice isolation)**
   - All features reside in one monolithic app, with no API gateways. frontend/backend are in one Node.js server. No service isolation.



h. **Lack of logging/monitoring mechanisms**
   - Suspicious actions like failed login and SQL injection are logged in frontend but not logged by the backend, no terminal output, no security log thus not able to monitor it.





i. **Absence of secure development practices (hardcoded secrets, missing tests)**
   - Keys are bundled with the app, once the source code is leaked any one can generate valid JWTs or decrypt tokens.

Part 2: Design and Code Fixes

1. **There's no limit to failed password attempts.**
   a. **Issue:** Attackers can make and use automatic login attempts to try possible passwords.
   b. **Fix:** locked out account after 5 attempts or display CAPTCHA on repeated failures.

c. **Code:**

```
const rateLimit = require('express-rate-limit');
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // limit each IP to 5 login requests
  message: 'Too many login attempts. Try again later.'
});
app.post('/rest/user/login', loginLimiter, loginHandler);
```

2. **Input's of users are not sanitized.**
   a. **Issue:** Attackers can inject scripts that lead to session hijacking or data theft.
   b. **Fix:** validate, sanitize and escape user inputs before using in the code.
   c. **Code:**

```
const userInput = req.query.q || '';
const sanitizedQuery = () => {
        return userInput.replace(/&/g, "&amp;").replace(/</g,
        "&lt;").replace(/>/g, "&gt;").replace(/"/g, "&quot;").replace(/'/g,
        "&#039;");
};
console.log(`<p>Results for: ${sanitizedQuery}</p>`);
```

3. **JWT secret is hardcoded in the source files.**
   a. **Issue:** Keys are readable on disk by any user/process and are not rotated or protected in prod.
   b. **Fix:** use environment-specific secret stores to store secrets and properly access control.
   c. **Code:**

```
require('dotenv').config();
const jwtSecret = process.env.JWT_SECRET;
const token = jwt.sign({ id: user.id }, jwtSecret, { expiresIn: '1h' });
```

Part 3: Reflection and Submission

1. **Which types of flaws did you find most concerning and why?**
   a. Flaws that I found most concerning are lack of input validation, since users can easily inject scripts or sql and readable secret keys are stored in source code that can lead to full system compromise.

2. **What secure design principles could have prevented these flaws?**
   a. The Secure design principles that could have prevented these flaws are ensuring the use of HTTPS and sanitation of any user's input. Making sure to restrict access in admin routes and using .env for storing critical info such as JWT secret is also important to take in mind.

3. **How can development teams integrate secure practices into their workflow?**
   a. They should regularly check for unsecured practices in the code and use tools or libraries that would help them secure the information that the system processes. They should also practice the team in using .env files and encrypting info to secure information.